

Assignment 10: Data Scraping

Prisha

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on data scraping.

Directions

1. Rename this file `<FirstLast>_A10_DataScraping.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Load the packages `tidyverse`, `rvest`, and any others you end up using.
 - Check your working directory

```
#1 Setup
library(tidyverse)
library(lubridate)
library(dplyr)
library(here); here()
```

```
## [1] "/Users/prisha/Desktop/EDE 2025"
```

```
library(rvest)
library(purrr)
```

2. We will be scraping data from the NC DEQs Local Water Supply Planning website, specifically the Durham’s 2024 Municipal Local Water Supply Plan (LWSP):
 - Navigate to <https://www.ncwater.org/WUDC/app/LWSP/search.php>
 - Scroll down and select the LWSP link next to Durham Municipality.
 - Note the web address: <https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024>

Indicate this website as the as the URL to be scraped. (In other words, read the contents into an `rvest` webpage object.)

```
#2 Scraping website
webpage <- read_html('https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024')
webpage
```

```
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
```

3. The data we want to collect are listed below:

- From the “1. System Information” section:
- Water system name
- PWSID
- Ownership
- From the “3. Water Supply Sources” section:
- Maximum Day Use (MGD) - for each month

In the code chunk below scrape these values, assigning them to four separate variables.

HINT: The first value should be “Durham”, the second “03-32-010”, the third “Municipality”, and the last should be a vector of 12 numeric values (represented as strings)“.

```
#3 Scraping
water_system_name <- webpage %>%
  html_node("td tr:nth-child(1) td:nth-child(2)") %>%
  html_text(trim = TRUE)

pwsid <- webpage %>%
  html_node("td tr:nth-child(1) td:nth-child(5)") %>%
  html_text(trim = TRUE)

ownership <- webpage %>%
  html_node("tr:nth-child(2) td:nth-child(4)") %>%
  html_text(trim = TRUE)

max_day_use <- webpage %>%
  html_nodes("th~ td+ td , th~ td+ td") %>%
  html_text(trim = TRUE)

water_system_name
```

```
## [1] "Durham"
```

```
pwsid
```

```
## [1] "03-32-010"
```

```
ownership
```

```
## [1] "Municipality"
```

```
max_day_use
```

```
## [1] "34.5000" "36.0600" "37.3300" "32.1000" "46.6500" "37.3600" "38.2000"
```

```
## [8] "41.9000" "36.5800" "36.7300" "42.9600" "34.4500"
```

4. Convert your scraped data into a dataframe. This dataframe should have a column for each of the 4 variables scraped and a row for the month corresponding to the withdrawal data. Also add a Date column that includes your month and year in data format. (Feel free to add a Year column too, if you wish.)

TIP: Use `rep()` to repeat a value when creating a dataframe.

NOTE: It's likely you won't be able to scrape the monthly withdrawal data in chronological order. You can overcome this by creating a month column manually assigning values in the order the data are scraped: "Jan", "May", "Sept", "Feb", etc... Or, you could scrape month values from the web page...

5. Create a line plot of the maximum daily withdrawals across the months for 2024, making sure, the months are presented in proper sequence.

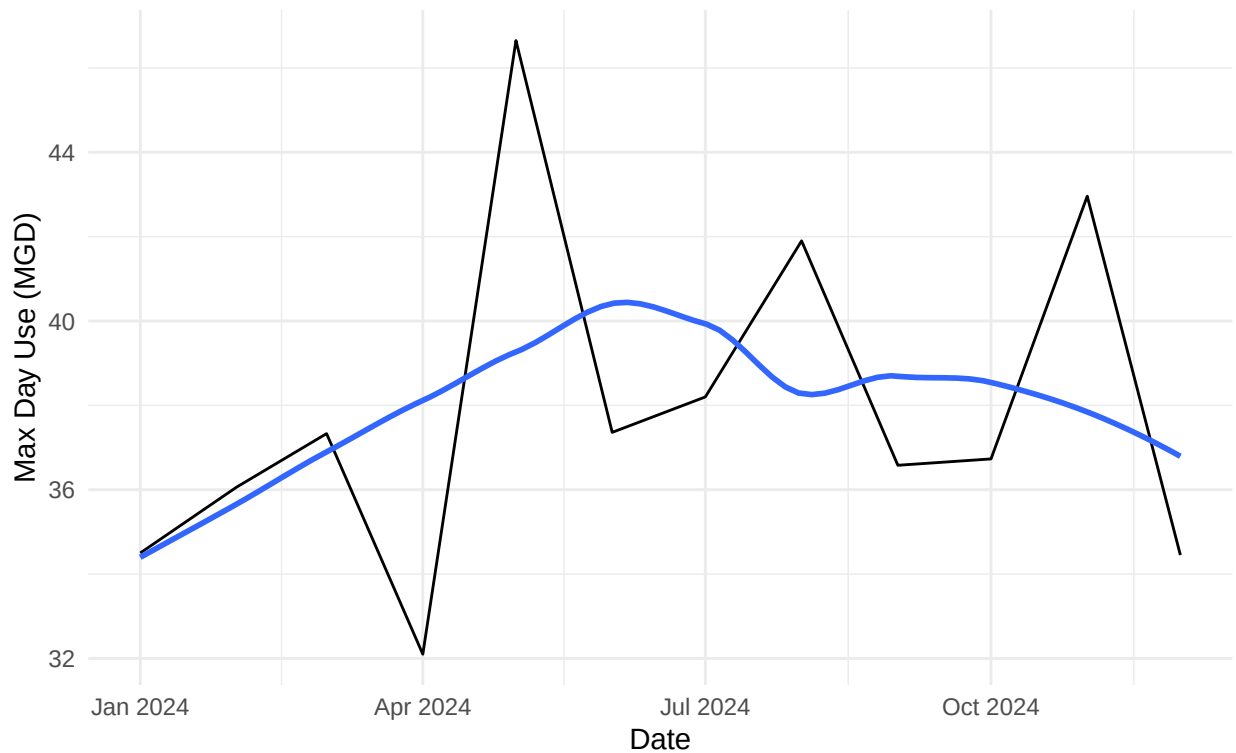
```
#4 Dataframe
df_webpage <- data.frame("Month" = rep(1:12),
                          "Year" = rep(2024,12),
                          "Max_Day_Use" = as.numeric(max_day_use))

df_webpage <- df_webpage %>%
  mutate(WaterSystemName = !!water_system_name,
         Ownership = !!ownership,
         PWSID = !!pwsid,
         Date = my(paste(Month, "-", Year)))

#5 Plot
ggplot(df_webpage, aes(x=Date, y=Max_Day_Use)) +
  geom_line() +
  geom_smooth(method="loess", se=FALSE) +
  labs(title = paste("2024 Max Day Use for", water_system_name),
       subtitle = ownership,
       y="Max Day Use (MGD)",
       x="Date") +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

2024 Max Day Use for Durham Municipality



6. Note that the PWSID and the year appear in the web address for the page we scraped. Construct a function with two input - “PWSID” and “year” - that:

- Creates a URL pointing to the LWSP for that PWSID for the given year
- Creates a website object and scrapes the data from that object (just as you did above)
- Constructs a dataframe from the scraped data, mostly as you did above, but includes the PWSID and year provided as function inputs in the dataframe.
- Returns the dataframe as the function’s output

```
#6. Automation
scrape_lwsp <- function(the_year, the_pwsid) {

  #Read the webpage
  the_website <- read_html(paste0(
    'https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=',
    the_pwsid, '&year=', the_year))

  #Use your working selectors
  the_system_name <- the_website %>%
    html_node("td tr:nth-child(1) td:nth-child(2)") %>%
    html_text(trim = TRUE)

  the_pwsid_text <- the_website %>%
    html_node("td tr:nth-child(1) td:nth-child(5)") %>%
    html_text(trim = TRUE)
```

```

the_ownership <- the_website %>%
  html_node("tr:nth-child(2) td:nth-child(4)") %>%
  html_text(trim = TRUE)

max_day_use <- the_website %>%
  html_nodes("th~ td+ td , th~ td+ td") %>%
  html_text(trim = TRUE) %>%
  as.numeric()

#Build a data frame
df_max_use <- data.frame(
  Month = 1:12,
  Year = rep(the_year, 12),
  Max_Day_Use_MGD = max_day_use
) %>%
  mutate(
    Water_System = the_system_name,
    PWSID = the_pwsid_text,
    Ownership = the_ownership,
    Date = as.Date(paste(Year, Month, "01", sep = "-"))
  )

return(df_max_use)
}

#Test
durham_2024 <- scrape_lwsp(2024, "03-32-010")
view(durham_2024)

```

7. Use the function above to extract and plot max daily withdrawals for Durham (PWSID='03-32-010') for each month in 2020

```

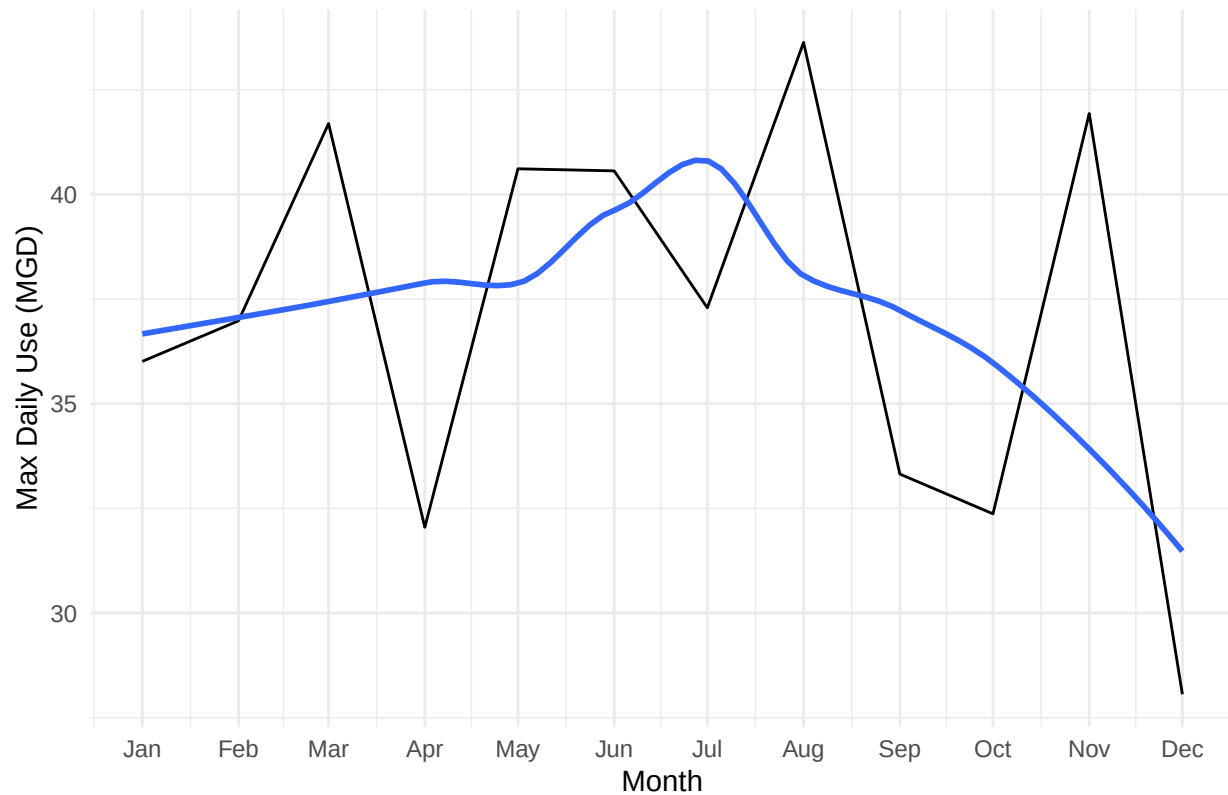
#7 Use of automation
durham_2020 <- scrape_lwsp(2020, "03-32-010")

ggplot(durham_2020, aes(x = Date, y = Max_Day_Use_MGD)) +
  geom_line() +
  geom_smooth(method="loess", se=FALSE) +
  labs(
    title = "Max Daily Withdrawals for Durham (2020)",
    x = "Month",
    y = "Max Daily Use (MGD)"
  ) +
  scale_x_date(date_labels = "%b", date_breaks = "1 month") +
  theme_minimal()

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Max Daily Withdrawals for Durham (2020)



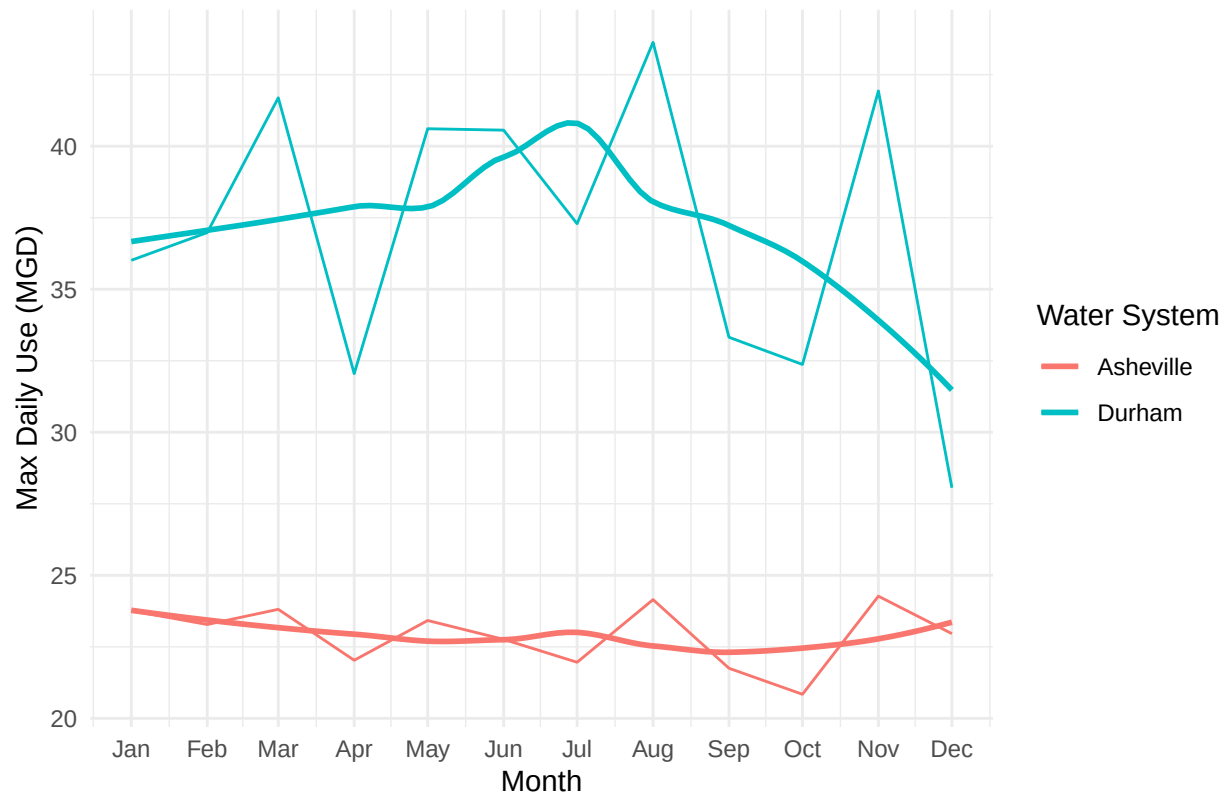
- Use the function above to extract data for Asheville (PWSID = '01-11-010') in 2020. Combine this data with the Durham data collected above and create a plot that compares Asheville's to Durham's water withdrawals.

```
#8 Combined data
asheville_2020 <- scrape_lwsp(2020, "01-11-010")
combined_data <- bind_rows(durham_2020, asheville_2020)

ggplot(combined_data, aes(x = Date, y = Max_Day_Use_MGD, color = Water_System)) +
  geom_line() +
  geom_smooth(method="loess", se=FALSE) +
  labs(
    title = "Comparison of Max Daily Withdrawals (2020)",
    x = "Month",
    y = "Max Daily Use (MGD)",
    color = "Water System"
  ) +
  scale_x_date(date_labels = "%b", date_breaks = "1 month") +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Comparison of Max Daily Withdrawals (2020)



- Use the code & function you created above to plot Asheville's max daily withdrawal by months for the years 2018 thru 2023. Add a smoothed line to the plot (method = 'loess').

TIP: See Section 3.2 in the "10_Data_Scraping.Rmd" where we apply "map2()" to iteratively run a function over two inputs. Pipe the output of the map2() function to `bindrows()` to combine the dataframes into a single one, and use that to construct your plot.

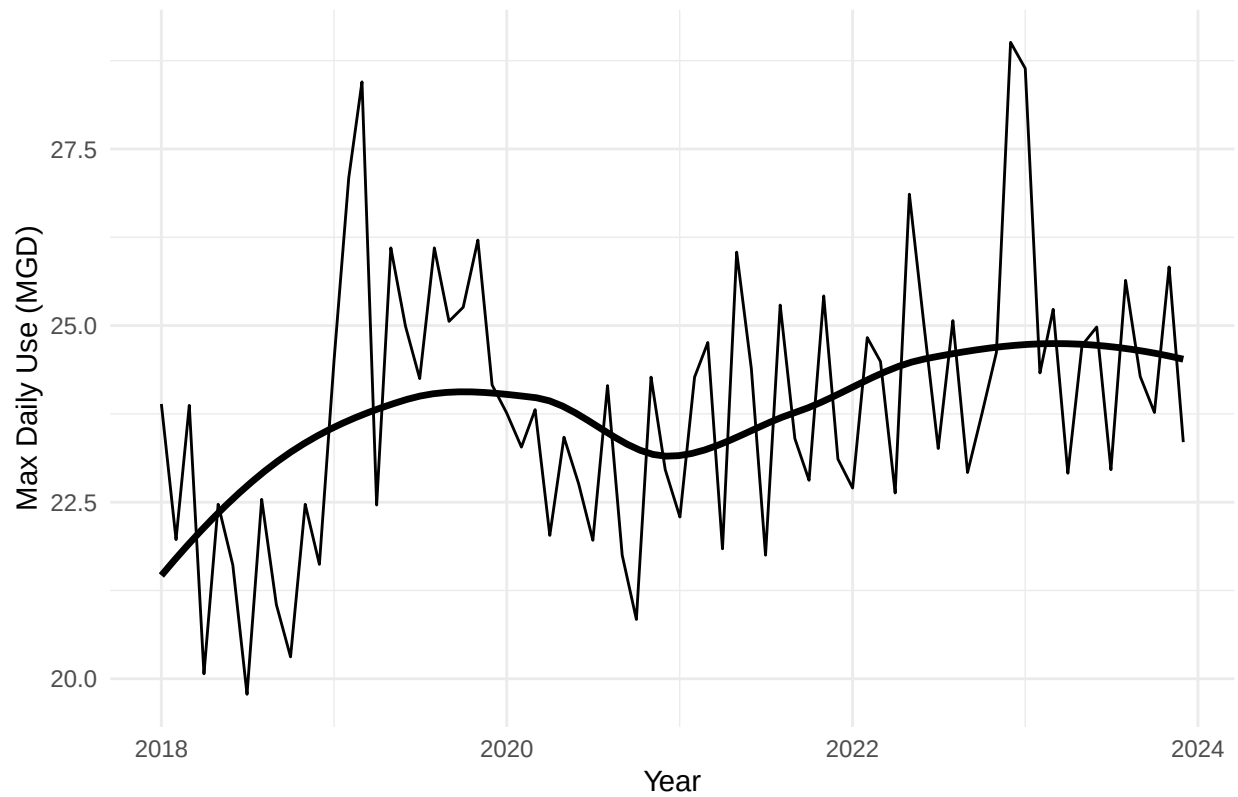
```
#9 Repeated automation
years <- 2018:2023
pwsids <- rep("01-11-010", length(years))

asheville_multi_year <- map2_dfr(years, pwsids, scrape_lwsp)

ggplot(asheville_multi_year, aes(x = Date, y = Max_Day_Use_MGD)) +
  geom_line() +
  geom_smooth(method = "loess", se = FALSE, color = "black", linewidth = 1.2) +
  labs(
    title = "Asheville Max Daily Withdrawals (2018-2023)",
    x = "Year",
    y = "Max Daily Use (MGD)"
  ) +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

Asheville Max Daily Withdrawals (2018–2023)



Question: Just by looking at the plot (i.e. not running statistics), does Asheville have a trend in water usage over time?

Answer: Yes, just by looking at the plot, Asheville appears to show a slight upward trend in water usage from 2018 to 2023. The smoothed loess line suggests an overall gradual increase in maximum daily withdrawals over time, even though there's noticeable month-to-month variability.